EX. NO. 03 - WIRESHARK NETWORK TRAFFIC ANALYSIS

Experiment Name	Wireshark Network Traffic Analysis
Aim	To capture and analyse network packets using Wireshark and identify HTTP traffic and POST requests from the cap
Tool Used	Wireshark, Windows, Wi-Fi Network Interface

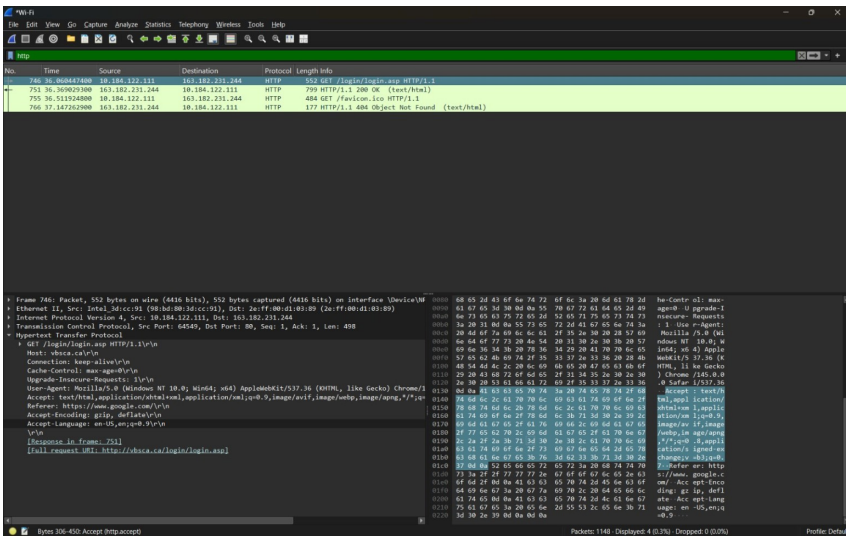
Experiment Focus: Network packet capture, HTTP traffic filtering, HTTP request identification, POST request analysis, and packet-level inspection using Wireshark.

OBJECTIVE

- Capture network traffic using Wireshark.
- Identify different network protocols.
- Filter HTTP packets using the **http** display filter.
- Analyse HTTP requests and responses.
- Examine an HTTP POST request and its packet details.

PROCEDURE AND OUTPUTS

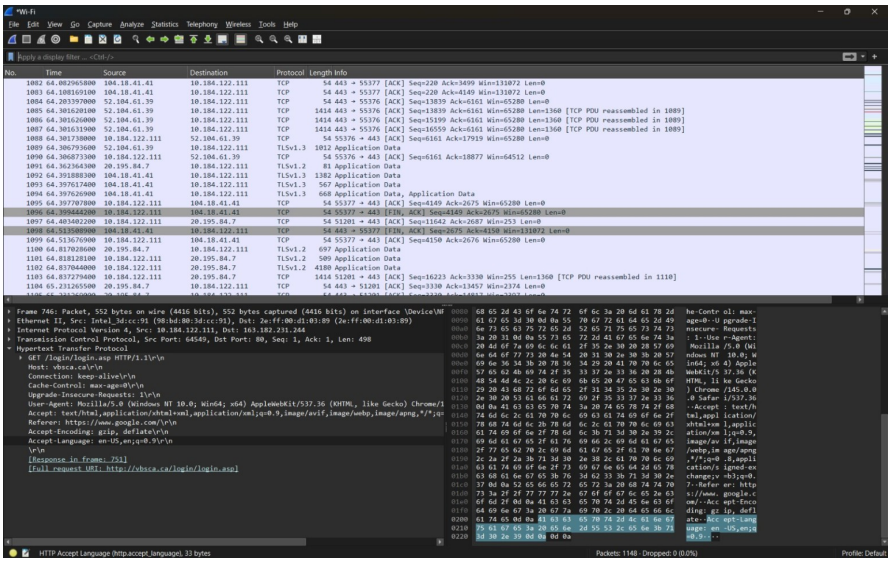
Output 1 - HTTP Traffic After Applying the Display Filter



The Wireshark display filter **http** was applied to the captured traffic. The filtered packet list isolates HTTP communication and shows the source, destination, protocol, packet length, and request information. This output provides the basis for selecting an HTTP request for detailed analysis.

Forensic significance: The filter view confirms that HTTP packets were successfully isolated from the captured network traffic. It also provides a compact view of the relevant HTTP communication for further forensic examination.

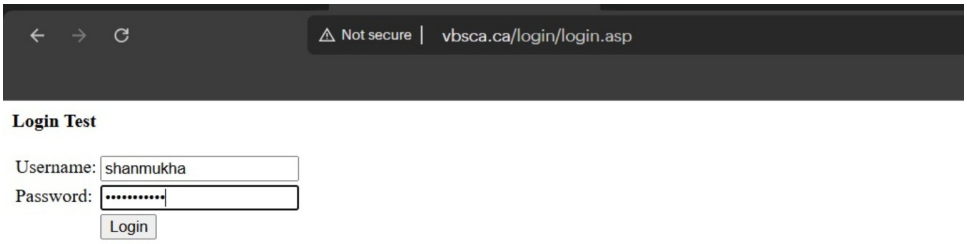
Output 2 - HTTP POST Request and Packet Details



The selected packet was examined at multiple protocol layers. The packet details show Ethernet, IPv4, TCP, and HTTP information, including a GET request to **/login/login.asp**, the host **vbs ca.ca**, browser-related headers, and the complete request URI. The packet bytes were also available for inspection.

Forensic significance: This output demonstrates packet-level analysis of an HTTP request. The combination of protocol fields, request headers, URI information, and packet bytes provides useful evidence for understanding the communication between the client and web server.

Output 3 - HTTP Login Page Used During Traffic Analysis



The browser view shows the HTTP login page associated with the captured communication. The page contains a username field, password field, and Login button. This visual output provides application-level context for the HTTP request observed in Wireshark.

Forensic significance: The login page helps correlate the captured network traffic with the corresponding web application activity. When examined together with the Wireshark packet details, it provides context for the HTTP request generated by the client.

OBSERVATION

Wireshark successfully captured network packets from the selected Wi-Fi interface. The **http** display filter was used to isolate HTTP communication. The captured traffic was then examined to identify HTTP requests and inspect their protocol-layer information. The supplied outputs show HTTP communication associated with a login page and detailed packet information.

RESULT

Wireshark was successfully used to capture and analyse network traffic, filter HTTP packets, identify an HTTP request, and inspect its Ethernet, IPv4, TCP, and HTTP details. The observed traffic included a request to **/login/login.asp**, and the related login page was available for application-level correlation.

CONCLUSION

The experiment demonstrated the practical use of Wireshark for network traffic analysis and digital forensic investigation. Filtering, packet selection, protocol-layer inspection, HTTP request analysis, and application-level correlation can be used together to understand network communication and identify relevant forensic artefacts.

Key forensic keywords
Wireshark | Network Traffic | Packet Capture | HTTP | HTTP Filter | POST Request | GET Request | Ethernet | IPv4 | TCP | HTTP Headers | Request URI | Packet Details | Packet Bytes | Digital Forensics